Error handling

PAGE NO.....
DATE / /

→ They allow you to handle errors gracefully instead of letting your program crash.

try Block:

→ The code inside the try block is executed.

→ If no errors occur, the except block is skipped.

except block

→ If an errors in the try block, Python stops executing the try block and jumps in the except block

→ The except block contains code to handle the error.

Uses:

- prevent crashes
- Graceful Error Handling.
- Debugging.

Syntax:

```
try:
    risky_code()
except ErrorType:
    handle_error()
```

PAGE NO.....
DATE / /

Common Error Types:

1) Value Error:
- Raised when a function receives an argument of the correct type but an inappropriate value (e.g. int("hello")).

2) Zero Division Error:
- Raised when dividing by zero (e.g. 10/0).

3) Type Error:
- Raised when an operation is performed on an inappropriate type (e.g. "hello" + 5).

4) File Not Found Error:
- Raised when trying to open a file that doesn't exist.

5) Indexed Error:
- Raised when trying to access an index that doesn't exist in a list (e.g. my_list[10]).

Finally Block

⇒ Executes always whether an error occurs or not. It's useful for cleanup tasks (e.g. closing files).

```
try:
    num = int(input("Enter a number: "))
    result = 10/num
except ValueError:
    print("Invalid input!")
except ZeroDivisionError:
    print("Error: Cannot divide by zero").
else:
    print(f"Result: {result}").  # Executes if no error occurs
finally:
    print("Execution complete.")
```